Assignment 1—Create an example-tracing tutor 05-832 / 05-432

Personalized Online Learning, Fall 2026

Due: Fri, Sep 4, 2026, 5pm

You are asked to design and implement an example-tracing tutor for dose calculation problems in veterinary medicine. This kind of calculation is an important part of the daily practice for "vet techs" who work in veterinary clinics or animal shelters. It can be challenging to learn.

You will use the Cognitive Tutor Authoring Tools (CTAT). No programming is needed. Key goals in the homework are to learn to build example-tracing tutors and to gain initial experience with a design process for intelligent tutoring systems. You will extend your tutor in Assignment 2.

A general description of how dose calculation can be done is given in a separate file, from the book Bill, R. (2009). *Medical mathematics and dosage calculations for veterinary professionals*. John Wiley & Sons.

Grading

Grading criteria include:

- Is your tutor well-designed?
  - Is the problem statement displayed in the tutor interface?
  - Are the subgoals and steps laid out clearly in the interface? Is the order of the steps clear?
  - Does the interface provide appropriate scaffolding for students learning the task, without "over-scaffolding"?
  - Is it clear to the student with what precision (in terms of the number of decimals) the solution steps should be entered?
  - Are the hints clear and informative?
  - Does the interface make good use of screen real estate (e.g., scrolling is kept to a minimum or avoided altogether, the hint panel never scrolls out of view).
- Does your tutor function correctly?
  - Does it support one complete solution path for the problem? Note: *There is no requirement that your tutor supports multiple solution paths, even if you created an interface that allows for multiple ways of solving.*
  - Does it accept steps with the indicated level of precision?
  - Does the tutor provide explanatory hints for all steps?
  - Is the tutor appropriately flexible with respect to the order of steps?
  - Is *all* problem-specific information (e.g., the problem statement, images) part of the behavior graph, and not the interface itself?
- Does the report include all required information?

Study the problem type for which to build a tutor

Familiarize yourself with the problem type and its solution strategies. Read the relevant material from *Medical mathematics and dosage calculations for veterinary*

professionals, available as a separate download from Canvas. You might study the example problem solution available on Canvas, and/or solve the problem yourself, using different strategies. It may help to use an online dosage calculator such as https://www.omnicalculator.com/health/dosage or https://www.vet-ebooks.com/vetdrugslist/drug-dose-calculator/ as well as an online unit conversions calculator such as that by Google.

Initial cognitive task analysis

Do a cognitive task analysis to gain insight into the solution strategies students may use and the main difficulties they may encounter as they learn to calculate dosages. Ask two people to solve two (or more) problems on paper. It is best to ask people for whom these problems may be challenging. Ask them to continuously "think aloud" (i.e., verbalize all their thoughts) while they do so. Say "Please keep talking" when they fall silent for more than a few seconds. Take detailed notes about their thought process. Collect their notes. Based on your observations and your participants' written solutions, identify a solution strategy that your tutor will support.

Design and implement your tutor

Now create your tutor. *You may restrict your tutor to the specific dosage problem given on Canvas and a single strategy within that problem,* even if in general this type of problem can be solved with multiple different strategies. In more detail:

- **Create a new package on Tutorshop** – A package is a file tree, needed for a tutor built with CTAT. You will learn in class how to create a package.
- **Create an interface for your tutor** – Design an interface that supports a step-by-step solution process based on the strategy you identified in your cognitive task analysis. We recommend designing it on paper first. Implement it with CTAT's Interface Editor or by writing HTML code by hand, using CTAT's interface components. Follow the Interface Design Principles (see below). Important: *All problem-specific information (e.g., problem statement, images) must be stored in the behavior graph, not in the interface itself.* Otherwise, the interface cannot be used for multiple problems. You can test this requirement by launching the interface with an empty behavior graph – the interface should only show information that is shared across all problems.
- **Create a behavior graph** – Create a behavior graph for your tutor, by demonstrating a solution using your tutor interface. Your behavior graph only needs to have a single path, even if the problem can be solved in multiple different ways.
- **Add hints** –Attach hints to all steps. For each, write multiple levels of hints with increasingly specific guidance. The hints should explain in the student's language why the step is the way it is. Keep the hints short and concise. See the Hint Design Principles below and the example hints.
- **Make sure the order of steps is appropriately flexible** – Determine how flexible you want the order of steps to be, by using ordered and unordered groups of links within your behavior graph.

- **Accommodate round off**[1] – Make sure that the student using your tutor knows with what precision values for the different quantities should be entered into the tutor interface (i.e., how many decimal places are required). This information should be provided in the tutor interface.

  Optional: A "deluxe version" of dealing with round off would be to allow for flexibility in terms of the precision with which the student enters a particular quantity – e.g., by letting the student enter "overly precise" numbers and then (to avoid problems in later steps) having CTAT reduce the precision. This feature could be implemented for example by using CTAT's feature for range matches combined with CTAT's input substitution feature. This way, it is predictable what value is used for calculations in later steps.

What to turn in

Please submit your tutor plus a report of *no more than 3 double-spaced pages* (content stretching beyond 3 pages will not be read). The tutor should be submitted on Tutorshop, the report on Canvas under Assignments > Assignment 1. Please make sure the report includes the following items:

1) The name of the Tutorshop package for your tutor. Please make sure the package contains all files that are needed to run your tutor, so the instructor can run your tutor for grading.
2) Your analysis of the task domain
   a) What solution strategy or strategies did you observe as you watched people solve the task?
   b) What are the main difficulties you saw?
   c) What strategy did you decide to implement in your tutor?
   d) What was the main reason for choosing this strategy?
3) Your tutor
   a) Show a screenshot of a complete problem in the tutor
   b) Discuss briefly how the tutor is designed to help students with the main difficulties that you identified in b.2, listed above
   c) Describe how you made the order of steps appropriately flexible.
4) Brief answers to the following questions:
   a) What was most difficult about doing this assignment? (Describe in 2 sentences.)
   b) How much time did you spend?
   c) One thing that was good about working with CTAT
   d) Two things that could be improved in CTAT

Please upload to Canvas under Assignments > Assignment 1.

[1] Dealing with round-off is needed in many task domains – it is therefore a realistic challenge. For example, in geometry, round off challenges occur when using pi.

Tutor Interface Design Principles

- Provide a clear and concise statement of the problem.
- Communicate the structure of the problem, make subgoals explicit using captions
- Break the problem into steps
- Order the steps so they follow the student's problem-solving structure
- Make the order of steps clear in the interface
- Provide step-by-step instructions in the interface
- Choose an interface size (e.g., laptop or mobile) that is realistic
- Avoid scrolling if at all possible
- Make sure the hint panel is always in view (never scrolls out of view)
- Make sure the interface generalizes across different problems of the same type (i.e., provide open fields for *all* problem-specific information, such as Text Areas – so problem-specific information can be stored in the behavior graph)
- Help students identify key information needed to tackle the problem
- Avoid "busy work" –steps that may not contribute to learning (e.g., copying) for example by making the tutor take over such steps
- Make sure the student must do the key steps, as a way of avoiding over-scaffolding; the system should not provide key steps or information from which steps can easily be inferred
- (Optional) Have the student first decompose a complex problem into major steps/subgoals before actually executing the steps (e.g., by dragging subgoals into the right order)

Hint Design Principles

- Use the student's language
- Use jargon only when it is jargon the student needs to learn
- Have about 3 hint levels for each step (if you have many more hint levels, it may be a sign that the step is too big)
- Go from general to specific
    - Remind the student about the goal or subgoal they are working on (if non-obvious – it may not be useful to just repeat the prompt for the step
    - Explain what problem-solving principle applies, and how
    - Space permitting (it may not), explain how you know that the principle applies (e.g., how its "IF-part" is instantiated in the current problem)
    - Give a "bottom-out hint," that is, a hint that literally gives away the step
- Brevity over completeness

- Hard to say in general how to apply this principle – but see the example below.

Example hint sequence

A reasonable set of hints for the step of deciding on a common denominator in a fraction addition problem (say, 1/4 + 1/6):

1. What could you do with the fractions so that they have the same denominator?
2. Choose a denominator both denominators go into.
3. The least common denominator between both fractions is 12. Enter '12' in the highlighted field.

These three hint levels clearly prioritize brevity over elaboration. The hints don't really say *why* one needs a common denominator in the first place, or why one would prefer to use the *least* common denominator, or how one computes a least common denominator.

Those topics would need to be addressed outside this tutor problem set, for example in classroom instruction about fraction addition or in a separate tutor problem set for computing the least common multiple of two numbers.

If you tried to incorporate all that information in the hint hint sequence, it might end up looking like this:

1. You can only add fractions when they have the same denominator.
2. What could you do with the two given fractions so that they have the same denominator?
3. Choose a denominator both denominators go into.
4. Find a number that is a multiple of 4 and is also a multiple of 6. To make later computations easier, find the *smallest* such number.
5. To find the least common multiple of 4 and 6, find the factors of each number (2 and 2) for 4, and (2 and 3) for 6, drop any common factors so they are not repeated (2) and multiply the remaining factors (2, 2, and 3).
6. 2x2x3 = 12 The least common denominator between both fractions is 12.
7. Enter '12' in the highlighted field.

This new set of hint levels has a high risk of being tl;dr (aka high cognitive load). The shorter hint sequence (with three hint levels) may be more effective, even if it relies on the student filling in some of the missing information for themselves. (If the student is successful in doing so, that in itself might help learning.) A general point may be that perfect hint sequences are very hard to come by.